

ECONOMIC COMMISSION FOR LATIN AMERICA AND THE CARIBBEAN

Anatomy of a Language Model

Fernando Fetis Riquelme

Spring 2026



NACIONES UNIDAS

CEPAL

LECTURE 2

What this talk covers

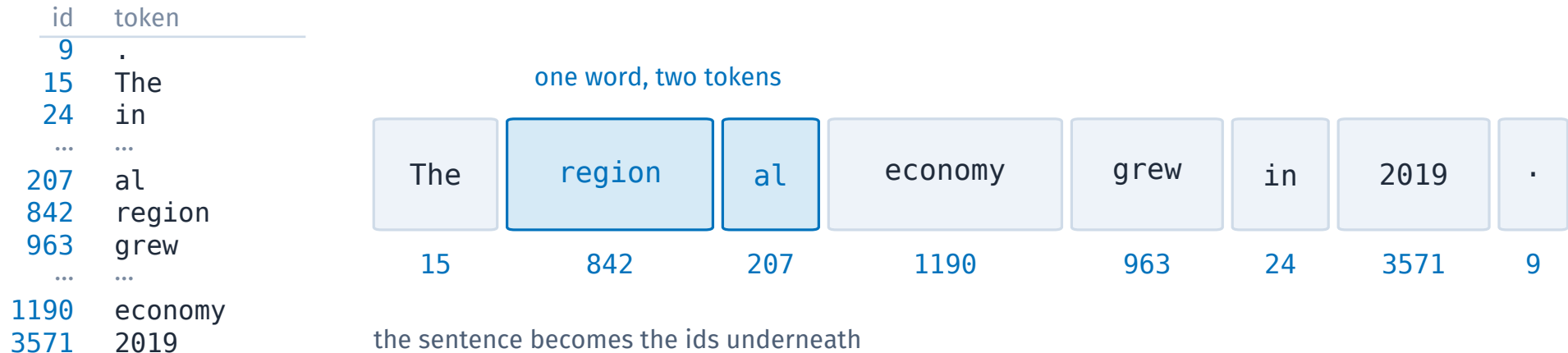
- 1 Tokens, training and generation
- 2 The Transformer
- 3 Pre-training and post-training
- 4 Scale, reasoning and evaluation
- 5 Closing

01

Tokens, training and generation

Tokenization

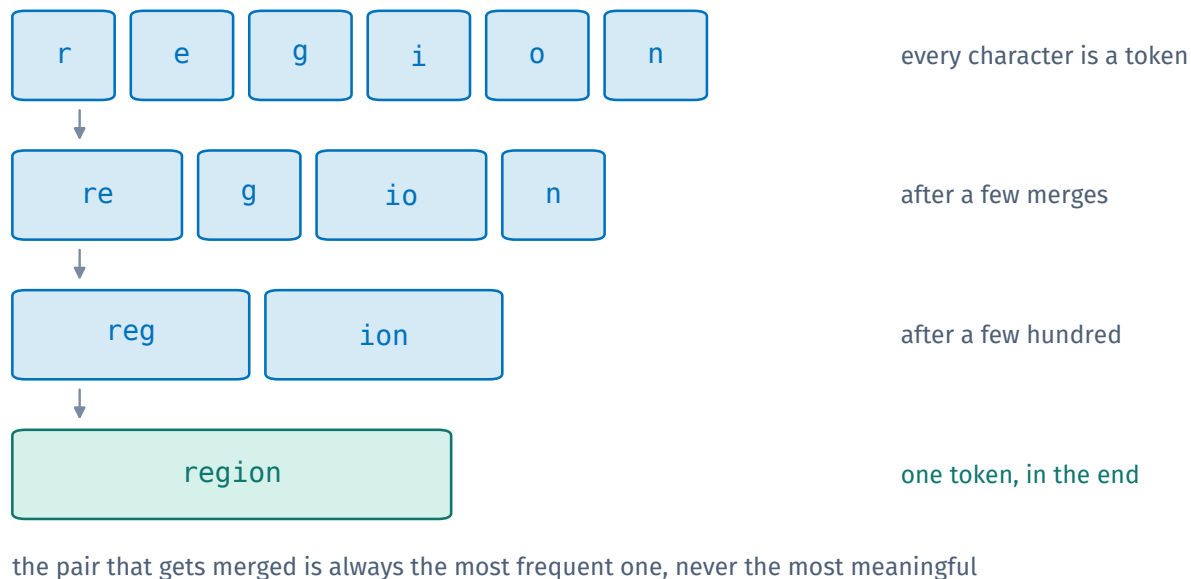
The text is transformed into a sequence of **tokens** belonging to a vocabulary:



The vocabulary is fixed before training and never changes. Everything the model will ever read or write is a sequence of **token IDs**.

Building the vocabulary

Initially, each character is an individual token. Then, all adjacent character pairs are counted, and the most frequent pair is combined into a new token.

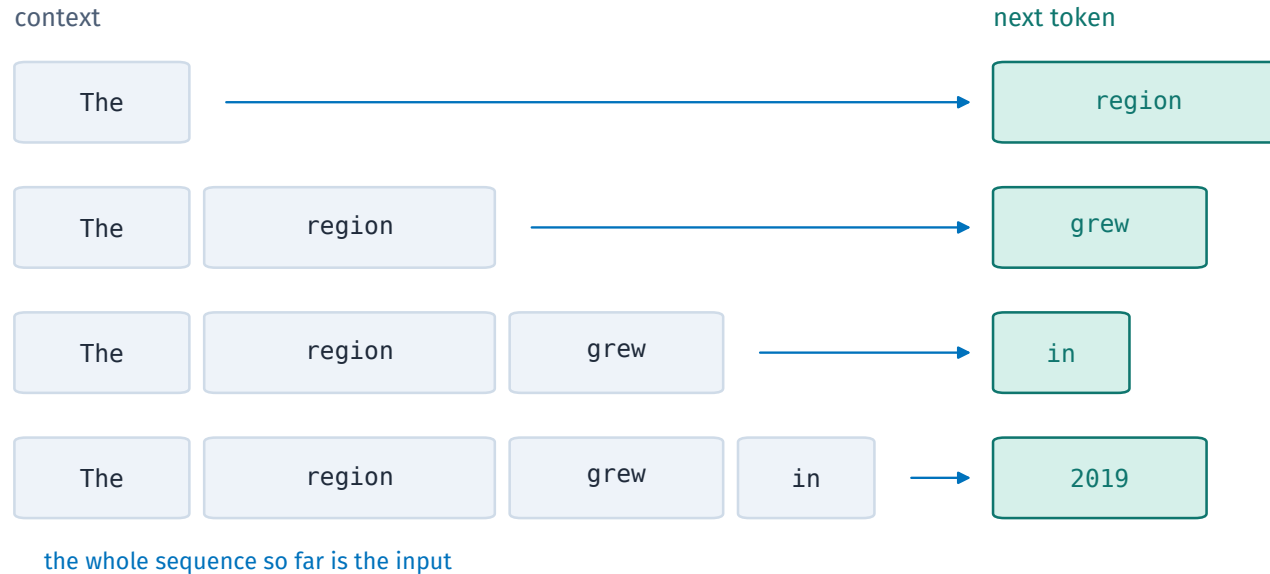


This process is repeated until the desired vocabulary size is reached.

The training objective

KEY IDEA

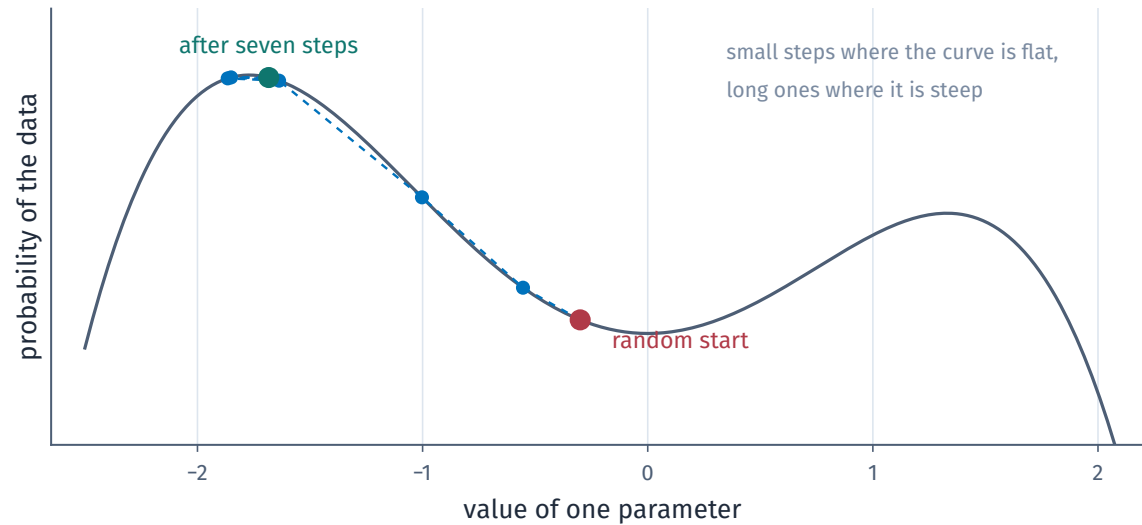
A **neural network** is trained to learn to predict the **next token**, given a sequence of tokens as **context**.



Training

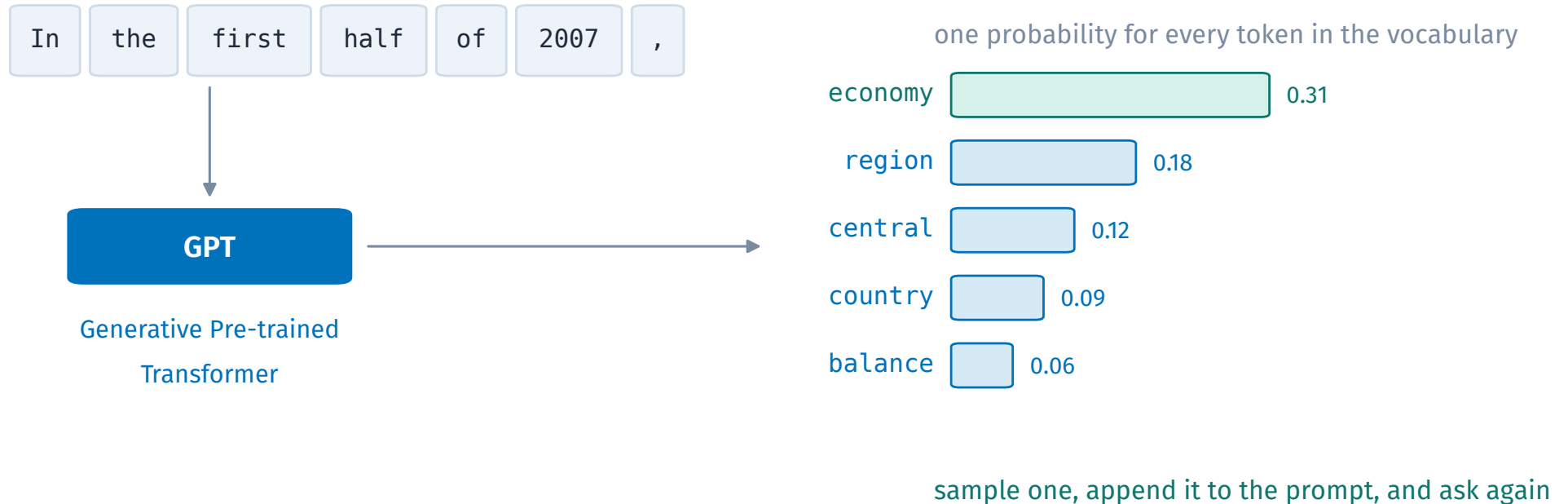
Therefore, the model $p_{\theta}(\text{next token} \mid \text{context})$ is trained so that the existing data, $\mathcal{D} = \{x_1, \dots, x_N\}$ has a high probability:

$$\max_{\theta} \frac{1}{N \cdot T} \sum_{n=1}^N \sum_{t=1}^T \log p_{\theta}(w_n^{(t)} \mid w_n^{(1, \dots, t-1)})$$



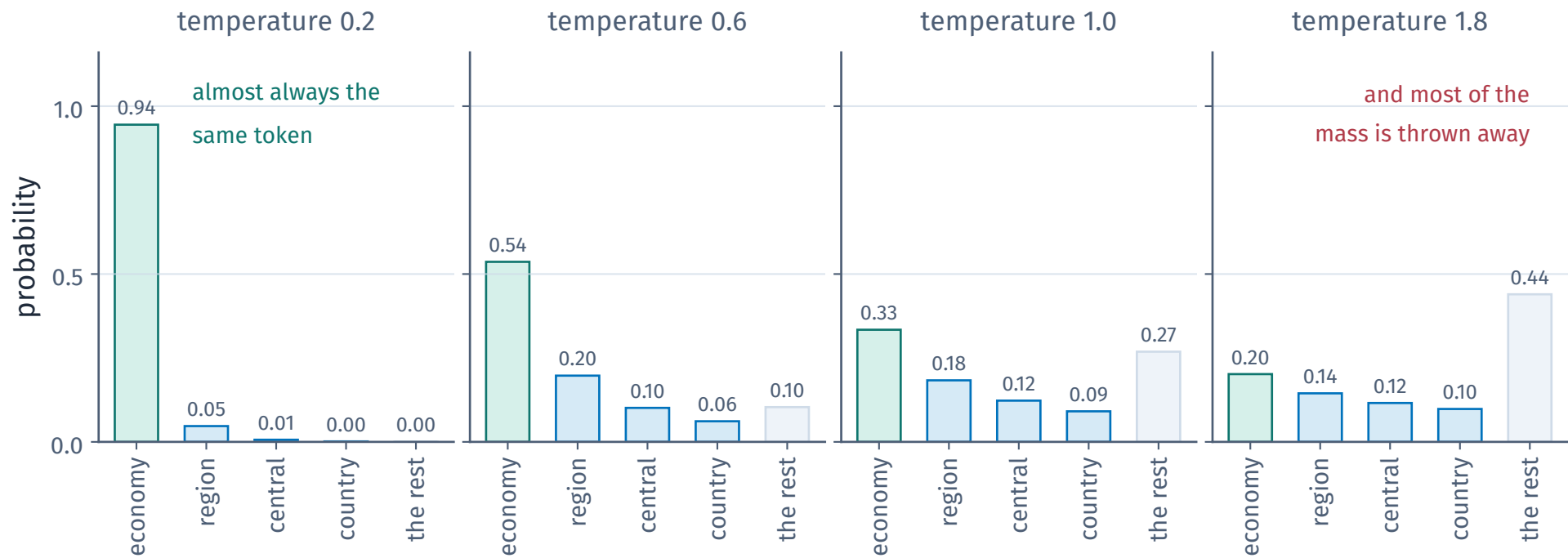
Autoregressive generation

To generate text, one begins with an initial text (**prompt**) and proceeds iteratively by predicting the next token using the trained model:



Decoding

A simple rule is to choose the **most probable token**. One could also choose from the K most probable tokens and adjust the **temperature** to control variability.

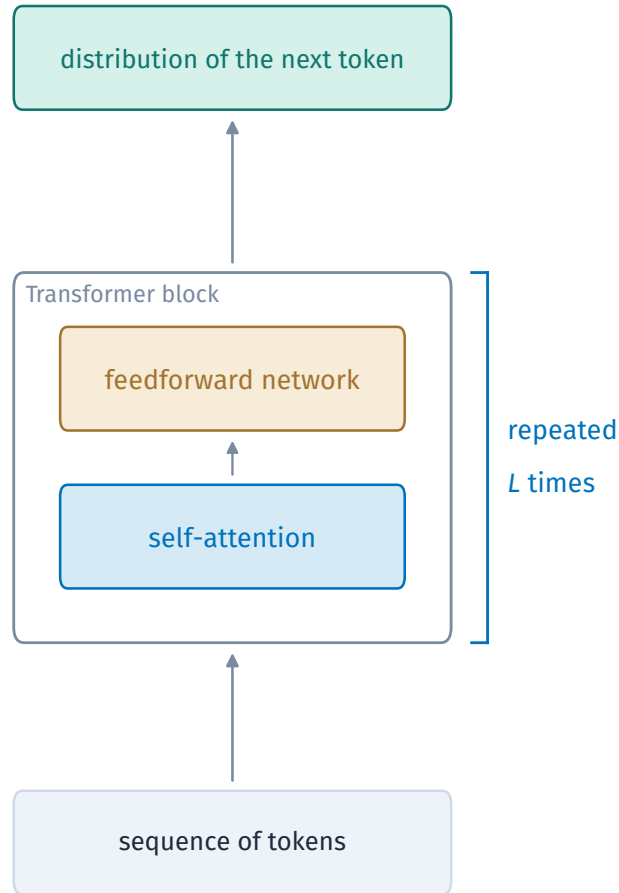


(Holtzman et al., 2020)

02

The Transformer

The Transformer



The **Transformer** is a neural architecture published by Google in 2017. Today, its logic is present in most modern models, including LLMs.

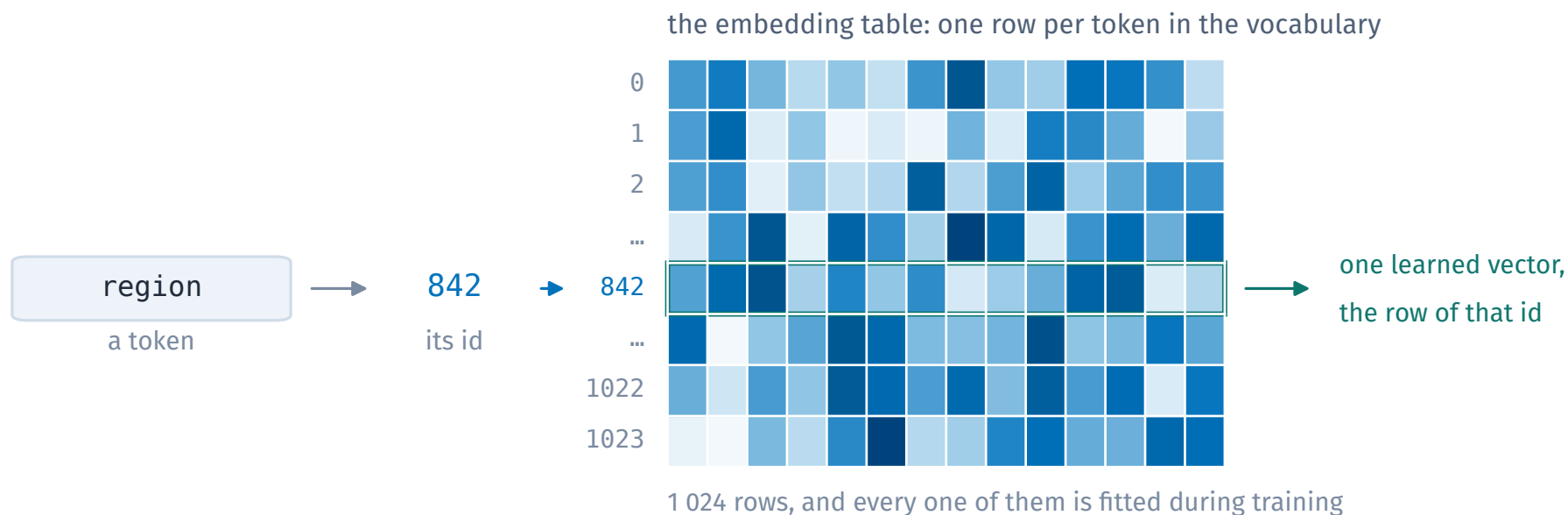
KEY IDEA

The architecture consists of a **Transformer block** that is repeated several times. To increase the capacity of these models, one can increase the number of blocks.

Here we will study the **GPT** architecture, which is a version of Transformer used to generate sequences.

Embeddings

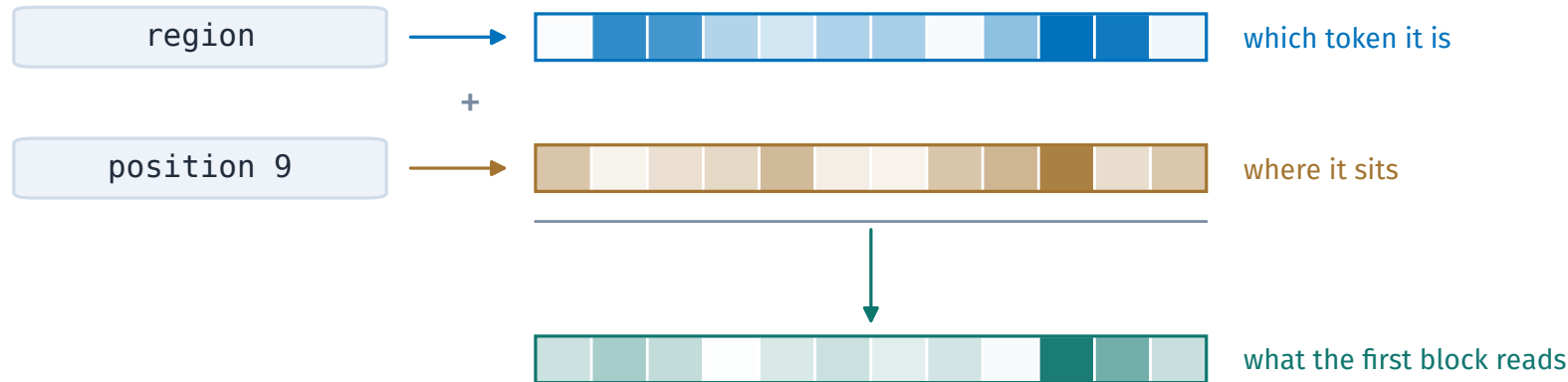
Each token ID is mapped to an **embedding vector**. In this way, the neural network can work with vectors, on which mathematical operations can be performed.



Positional encoding

A **positional embedding** table is learned in a similar way, where each position within the sequence has an associated vector.

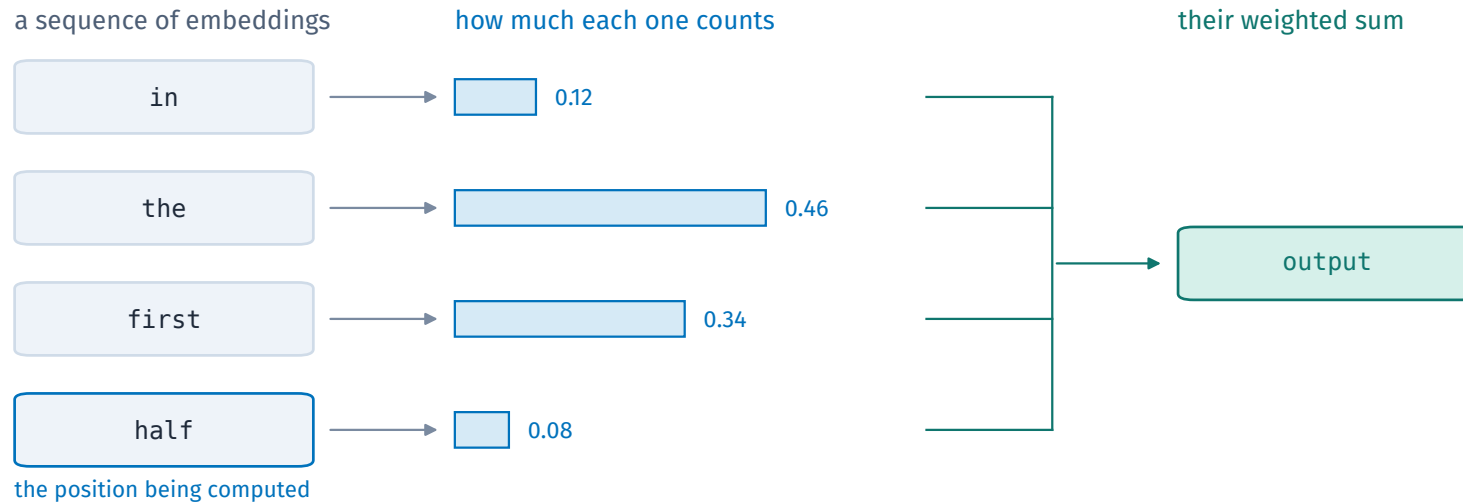
a position is a row of a second learned table, exactly like a token



every block then rewrites this sequence of vectors into a better one

The first Transformer block of the neural network receives the sum of the token and position embedding vectors for each token within the input sequence.

Self-attention



The output for token i is a weighted sum of the tokens up to it:

$$y^{(i)} = \sum_{j=1}^i \alpha_{ij} W_V x^{(j)}$$

The weight α_{ij} is assigned based on the similarity between tokens i and j :

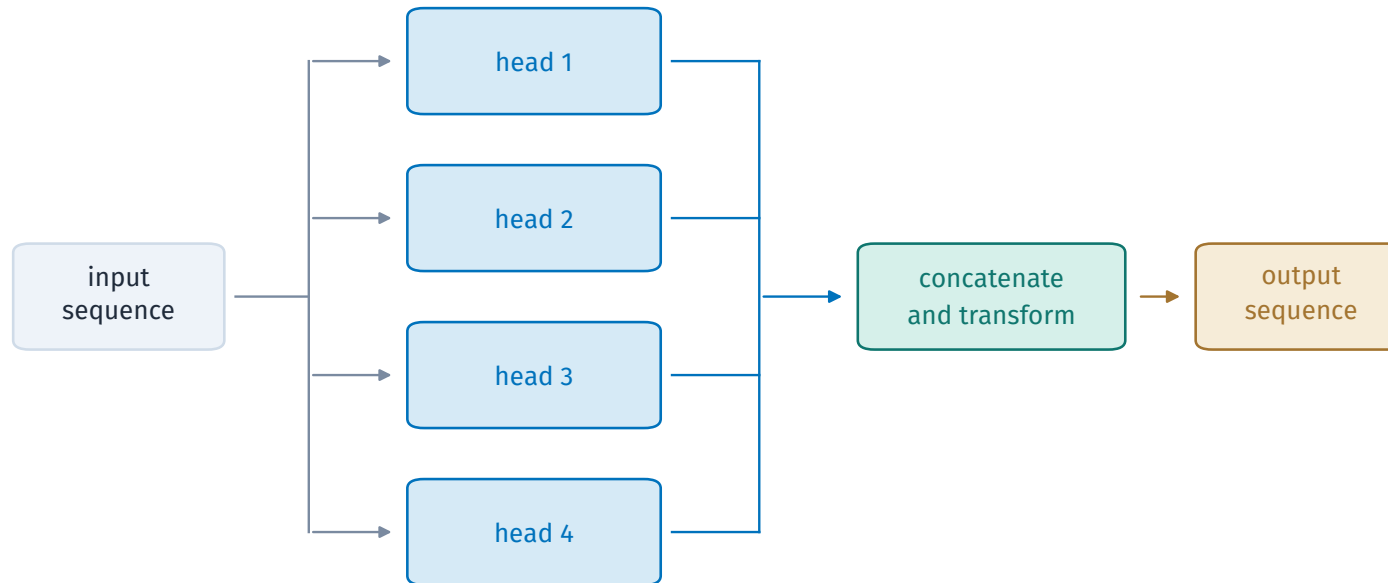
$$\alpha_{ij} = \text{softmax}_j \left(\frac{1}{\tau} W_Q x^{(i)} \cdot W_K x^{(j)} \right)$$

Multi-head attention

KEY IDEA

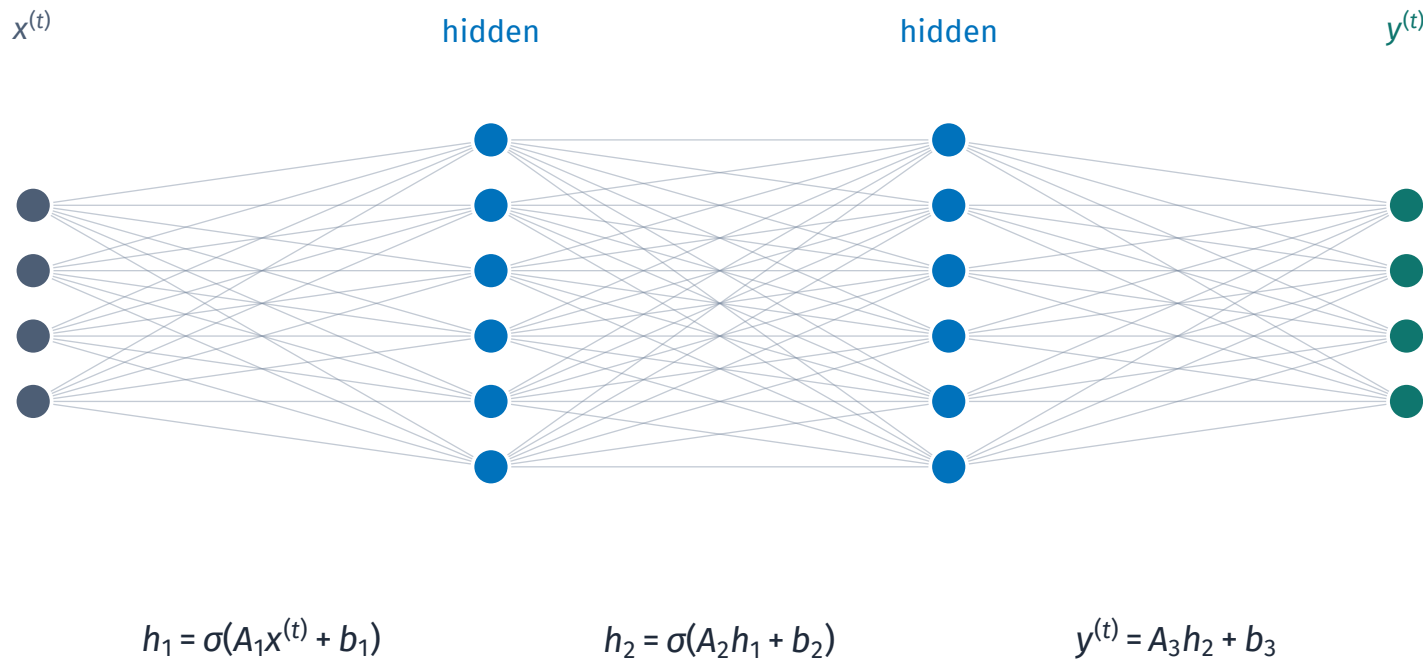
The aforementioned mechanism is called an **attention head**. An attention head typically learns specific subtasks.

It is common to use several heads in parallel and concatenate their outputs:



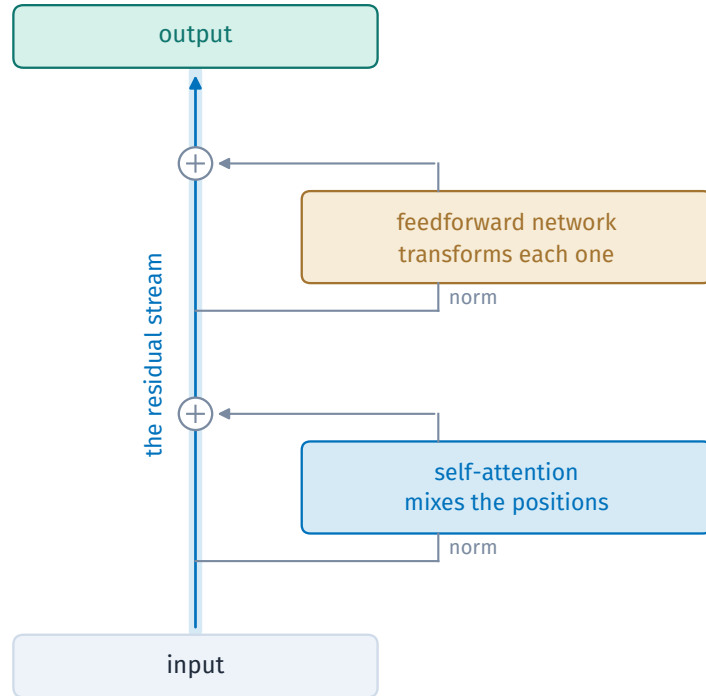
The feedforward network

The attention sub-block allows the sequence tokens to interact. The **feed-forward** sub-block transforms the resulting vector at each position.



The Transformer block

each part writes a correction onto the stream



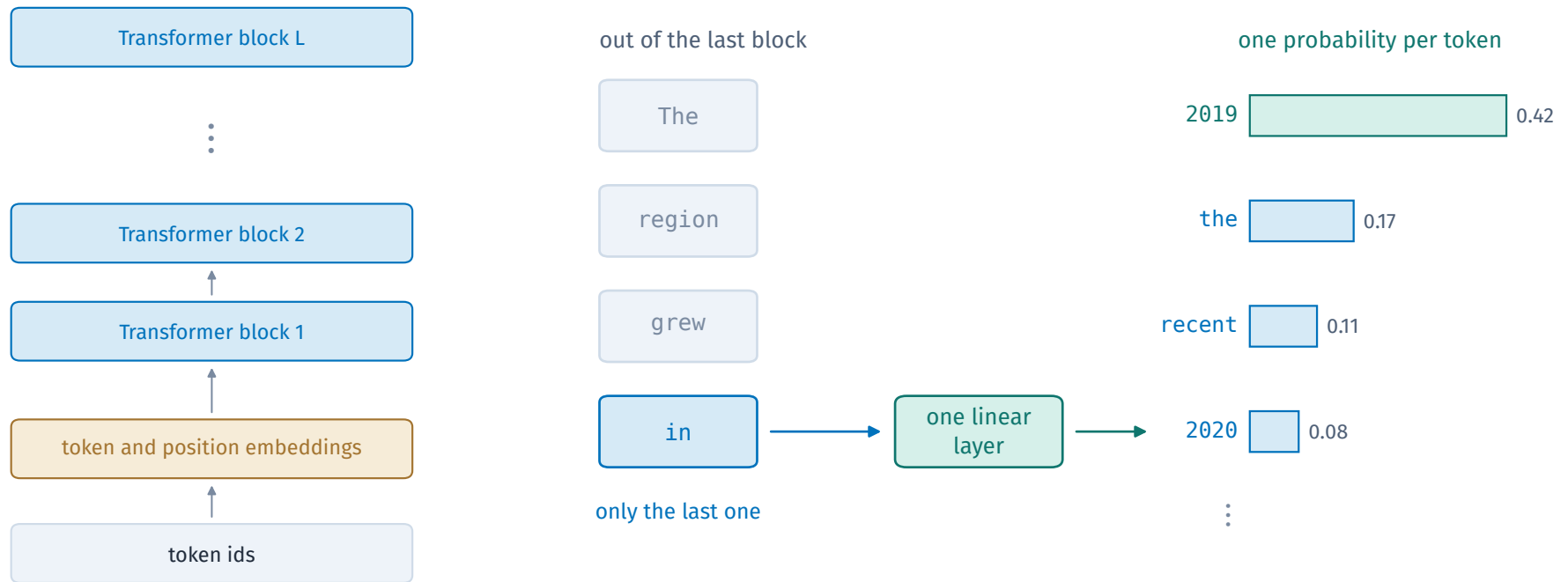
Consequently, a **Transformer block** serves as a backbone that transforms the vector representations of each token.

First, each position interacts with the other positions, and then each position is transformed individually.

Both sub-blocks operate by adding their contribution to the backbone.

The GPT architecture

The GPT architecture stacks multiple Transformer blocks and then transforms the final representation of the **last token** to predict the **probability of the next token**:

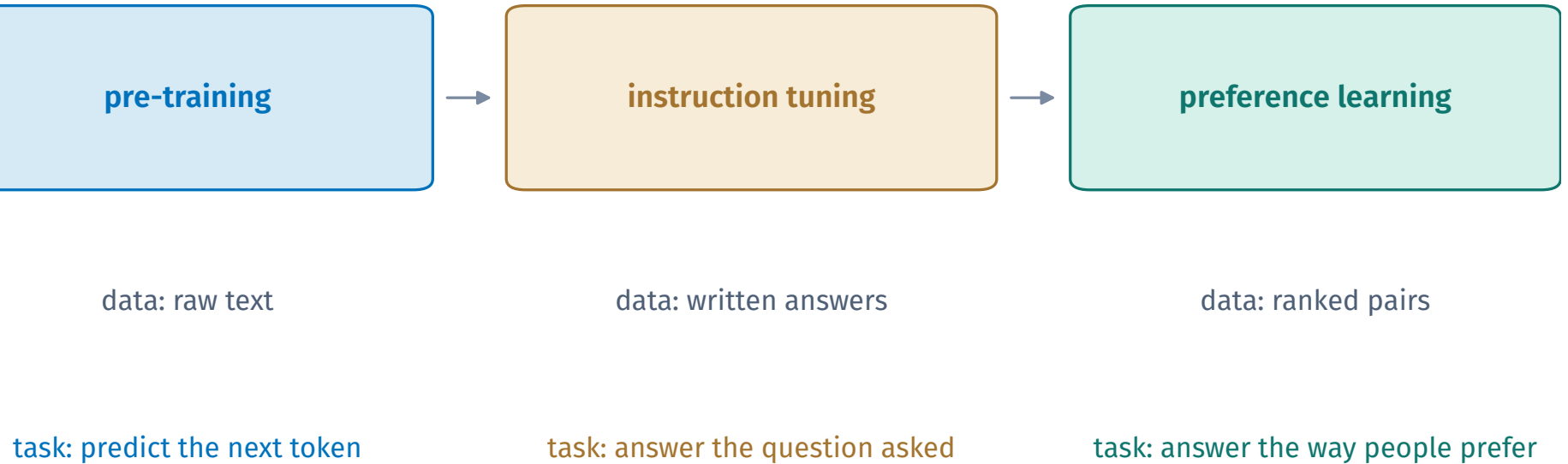


(Radford et al., 2018; 2019)

03

Pre-training and post-training

Training phases



Instruction tuning

Pre-training yields a **base model**, which is then **fine-tuned** to learn how to respond in a helpful style. Data following the desired format is used for this purpose.

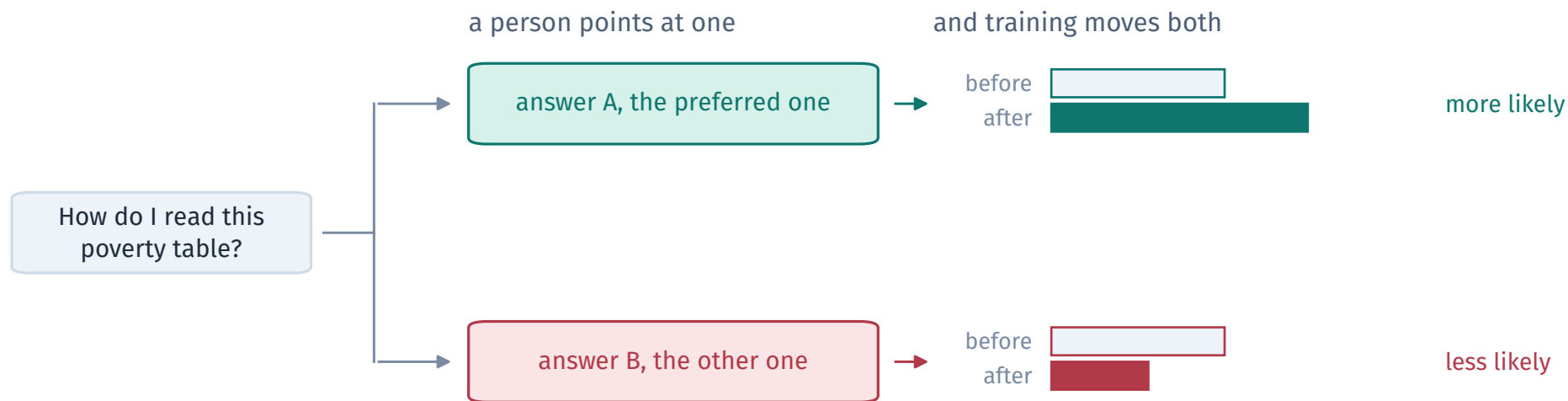


WATCH OUT

The model is trained to strictly adhere to its **system prompt**.

Preference learning

The next stage is the one associated with **alignment**, where the model is fine-tuned to generate preferred responses.



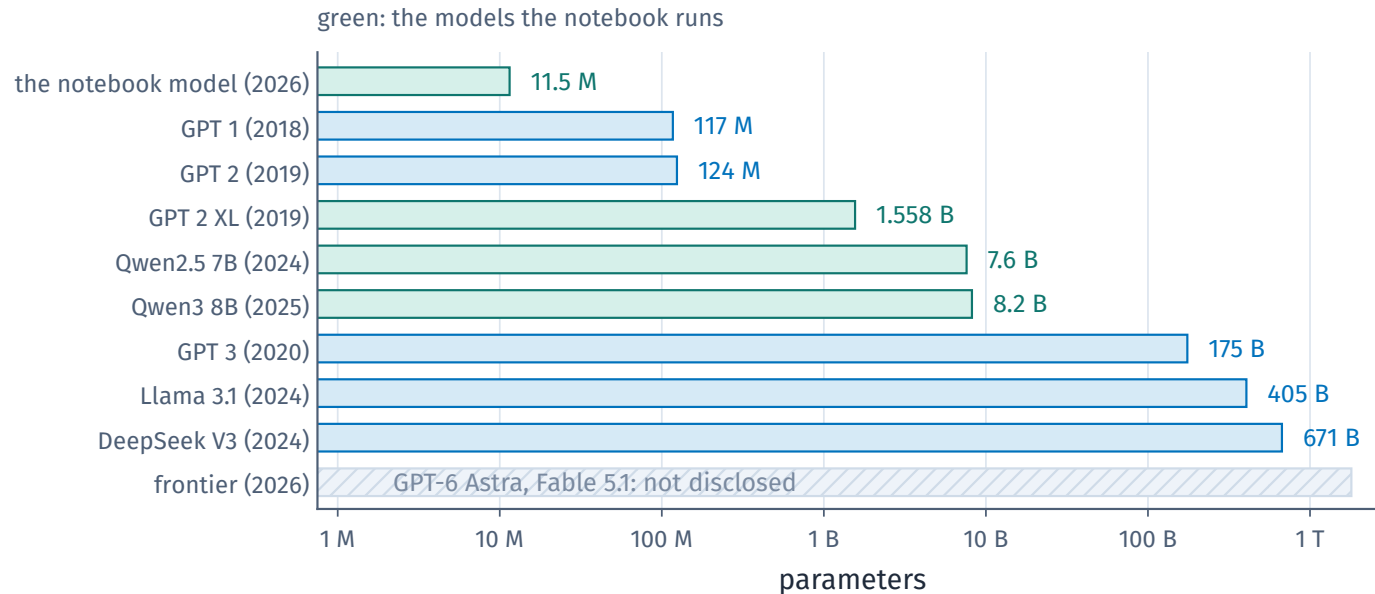
no one writes the ideal answer; someone only picks the better of two

04

Scale, reasoning and evaluation

Scaling laws

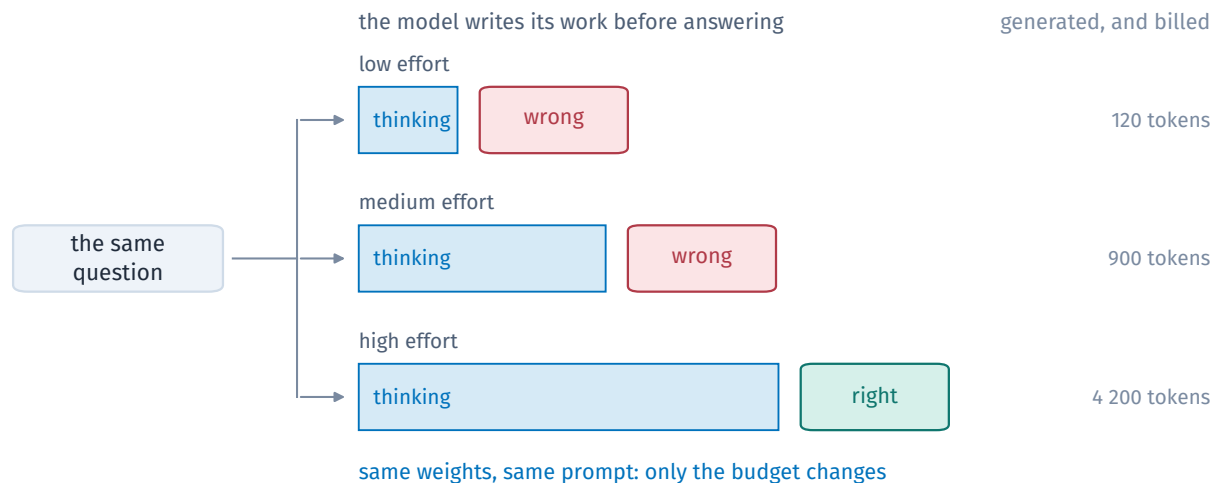
The error falls predictably as the model, the data and the compute grow, which is why the number of parameters kept climbing. Between GPT 1 and the models used today the general backbone barely changed.



(Grattafiori et al., 2024; Hoffmann et al., 2022; Kaplan et al., 2020)

Reasoning

While **Chain of Thought** began as a prompting technique, the model is now trained using this type of data to encourage it to reason through complex problems.



Test-time compute became a second scaling axis, where a harder question is answered by thinking longer, not by a larger model.

Evaluating a model

held-out perplexity

cheap, runs
during training

says nothing
about usefulness

public benchmarks

comparable
across models

the answers leak
into training data

a model as judge

scales to
thousands of answers

prefers long,
confident answers

people, on your task

the only measure
that decides

slow, and expensive
to repeat

05

Closing

Takeaways

- | | | |
|---|---|-----------|
| 1 | A model does not look anything up: it predicts the next token . Whatever it states as fact still has to be checked against a source. | section 1 |
| 2 | Different models bill or measure usage based on tokens, so extensive contexts or reasoning processes result in higher costs . | section 1 |
| 3 | All the model reads is just a long sequence of tokens. Special tokens are used to separate them into roles or subsections. | section 3 |
| 4 | Public scores do not transfer. Keep a small set of your own cases and run it again whenever the model or the prompt changes. | section 4 |

Next session

- 1 Other families: encoders, embeddings, diffusion, multimodal
- 2 RAG
- 3 From model to agents
- 4 Integration protocols: MCP and A2A
- 5 Use of Claude Code

Thank you

Lecture 2 · Anatomy of a Language Model



References

- Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., & Amodei, D. (2017). Deep Reinforcement Learning from Human Preferences. *Advances in Neural Information Processing Systems*, 30.
- DeepSeek-AI. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*.
- Grattafiori, A., Dubey, A., Jauhri, A., & Pandey, A. (2024). *The Llama 3 Herd of Models*.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., & Sifre, L. (2022). Training Compute-Optimal Large Language Models. *Advances in Neural Information Processing Systems*, 35.
- Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The Curious Case of Neural Text Degeneration. *International Conference on Learning Representations*.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). *Scaling Laws for Neural Language Models*.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., & Lowe, R. (2022). Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems*, 35.
- Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). *Improving Language Understanding by Generative Pre-Training*. OpenAI.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *Advances in Neural Information Processing Systems*, 36.
- Sainz, O., Campos, J. A., García-Ferrero, I., Etxaniz, J., Lacalle, O. L. de, & Agirre, E. (2023). NLP Evaluation in Trouble: On the Need to Measure LLM Data Contamination for each Benchmark. *Findings of the Association for Computational Linguistics: EMNLP 2023*.
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., & Le, Q. V. (2022a). Finetuned Language Models Are Zero-Shot Learners. *International Conference on Learning Representations*.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022b). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems*, 35.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Li, Z., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36.